

Instructions for Lab Submission

1. File Naming and Submission:

- Submit your solution as a **ZIP file** named `lab<number>.<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
- Ensure that the ZIP file contains:
 - C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
 - No unnecessary files (such as executables, temporary files, or editor configs).
- Do not submit **TEXT(.txt)** or **EXE(.exe)** files.

2. Code Organization:

- Write each task in its own `.cpp` file (as specified above).
- Ensure that your code is modular and logically separated into functions when specified.

3. Code Neatness:

- Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
- Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
- Add clear explanations where required (or specified)** to make the logic of your program easy to follow.

4. Code Compilation and Functionality:

- Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
- Double-check that the functionality of the program meets the requirements outlined in the task.

5. Additional Instructions:

- Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
- Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
- If you encounter any issues or require clarifications, reach out before the submission deadline.

6. Deadline:

- Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
- Double-check that the zip file is correctly named and contains all necessary files before submitting.

7. Plagiarism Policy:

- The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

Arrays

An **array** stores multiple values of the same data type under one variable name.

For example:

```
int marks[5] = {72, 85, 61, 90, 77};
```

The size of this array is **5**. Array positions are called **indices**, and indexing begins from **0**.

Index	0	1	2	3	4
Value	72	85	61	90	77

An individual element is accessed using its index:

```
cout << marks[0];    // 72
cout << marks[3];    // 90

marks[2] = 70;       // changes 61 to 70
```

For an array of size N , the valid indices are:

$$0, 1, 2, \dots, N-1$$

Accessing an index outside this range may cause incorrect behavior or a runtime error.

Traversing an Array

Traversal means visiting the elements of an array one at a time.

A **for** loop is commonly used when the array size is known:

```
const int SIZE = 5;
int numbers[SIZE] = {4, 7, 2, 9, 6};

for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << " ";
}
```

The loop variable **i** represents the current index.

A **while** loop can perform the same traversal:

```
int i = 0;

while (i < SIZE) {
    cout << numbers[i] << " ";
    i++;
}
```

During traversal, array elements can also be used to calculate values such as a total, minimum, maximum, or average.

```
int total = 0;

for (int i = 0; i < SIZE; i++) {
    total += numbers[i];
}
```

for and while Loops

Both **for** and **while** repeat a block of statements while a condition is true.

A **for** loop keeps its initialization, condition, and update together:

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";  
}
```

The equivalent **while** loop is:

```
int i = 0;  
  
while (i < 5) {  
    cout << i << " ";  
    i++;  
}
```

A **for** loop is convenient when the number of iterations is known. A **while** loop is useful when repetition is controlled by a condition and the number of iterations may not be known beforehand.

Nested Loops and Grids

A loop placed inside another loop is called a **nested loop**.

Nested loops are useful for working with rows and columns, such as a grid.

```
for (int row = 0; row < 3; row++) {  
  
    for (int column = 0; column < 4; column++) {  
        cout << "*" << " ";  
    }  
  
    cout << endl;  
}
```

Output:

```
* * * *  
* * * *  
* * * *
```

For every one iteration of the outer loop, the inner loop completes all of its iterations.

Algorithms on Arrays

An **algorithm** is a sequence of steps used to solve a problem. Many common array problems can be solved by repeatedly examining or rearranging array elements.

The following algorithms are written in pseudocode to focus on their logic.

Linear Search

Linear search examines array elements one by one until the target value is found or the array ends.

```
SET position = -1  
  
FOR i FROM 0 TO SIZE - 1  
  
    IF array[i] EQUALS target  
        SET position = i
```

```
        STOP THE LOOP
    END IF

END FOR
```

If `position` remains `-1`, the target was not found.

Linear search does not require the array to be sorted.

Shuffling an Array

Shuffling rearranges the elements of an array into a random order.

One common method starts at the last element and swaps it with a randomly selected element from the part of the array that has not yet been fixed.

```
SET i = SIZE - 1

WHILE i > 0

    GENERATE random index j FROM 0 TO i

    SET temp = array[i]
    SET array[i] = array[j]
    SET array[j] = temp

    DECREASE i BY 1

END WHILE
```

The range of the random index becomes smaller after each iteration.

Selection Sort

Selection sort repeatedly finds the smallest value in the unsorted part of the array and places it at the next correct position.

To sort in ascending order:

```
FOR i FROM 0 TO SIZE - 2

    SET minIndex = i

    FOR j FROM i + 1 TO SIZE - 1

        IF array[j] < array[minIndex]
            SET minIndex = j
        END IF

    END FOR

    SWAP array[i] WITH array[minIndex]

END FOR
```

After each outer-loop iteration, one more position at the beginning of the array is in its correct sorted location.

Bubble Sort

Bubble sort repeatedly compares neighboring elements and swaps them when they are in the wrong order.

To sort in ascending order:

```
FOR pass FROM 0 TO SIZE - 2
```

```

FOR i FROM 0 TO SIZE - 2 - pass
    IF array[i] > array[i + 1]
        SWAP array[i] WITH array[i + 1]
    END IF
END FOR
END FOR

```

After each complete pass, the largest remaining value moves toward the end of the array.

For example:

```

Start:      5  2  4  1
After Pass 1: 2  4  1  5
After Pass 2: 2  1  4  5
After Pass 3: 1  2  4  5

```

Selection Sort vs. Bubble Sort

Both algorithms use nested loops, but they rearrange values differently.

	Selection Sort	Bubble Sort
Main idea	Find the smallest remaining value	Compare neighboring values
Movement	Places one selected value into its correct position	Repeatedly swaps adjacent values
After a pass	A position at the beginning is fixed	A position near the end is fixed

Important Array Reminders

- Array indexing begins at 0.
- An array of size `SIZE` has valid indices from 0 to `SIZE - 1`.
- Use `i < SIZE`, not `i <= SIZE`, when traversing the complete array from index 0.
- `array[i]` refers to the value stored at index `i`.
- A single traversal can perform more than one operation, such as calculating a total while also searching for a value.
- Nested loops are commonly used when values must be compared with other values or when processing rows and columns.

Lab Tasks

Task 1: The Broken Score Analyzer.....

A teaching assistant has written a program to analyze student scores. The program should find the minimum score, maximum score, average score, and search for a particular score in the same traversal of the array.

Unfortunately, the program contains several syntax, logical, and runtime errors. A few parts of the program are also missing.

Problem Description

The program stores eight student scores in an array. It asks the user for a score to search for and:

- calculate the total of all scores,
- determine the minimum and maximum scores, and
- perform a search for the entered score.

If the target appears more than once, the program should report the index of its **first occurrence**.

The average must be displayed as a decimal value.

The following program does not work correctly:

```
#include <iostream>
using namespace std;

int main() {

    const int SIZE = 8;
    int scores[SIZE] = {72, 85, 61, 90, 77, 68, 85, 94};

    int target;
    cout << "Enter score to search: ";
    cin >> target;

    int minimum = scores[0];
    int maximum = scores[0];
    int total = scores[0];
    int position = -1;

    for (int i = 1; i <= SIZE; i++) {

        total += scores[i];

        if (scores[i] < minimum) {
            maximum = scores[i];
        }
        else if (scores[i] > maximum) {
            minimum = scores[i];
        }

        if (scores[i] == target) {
            position = i;
        }

        // A statement required to preserve the first match is missing here.
    }

    double average = total / SIZE - 1;

    cout << "Minimum Score: " << minimum << endl;
    cout << "Maximum Score: " << maximum << endl;
    cout << "Average Score: " << average << endl;

    // Write the complete code required to display whether the
    // target was found and, if found, its index.
```

```

    return 0;
}

```

Find and fix all syntax, logical, and runtime errors and complete the missing parts of the program.

You must keep the overall structure of the program and use only the existing **single for loop** for calculating the statistics and performing the search. Do not add another loop.

Task 2: The Broken Card Shuffler

A card game stores eight numbered cards in an array. Before the game begins, the cards must be shuffled into a random order.

A programmer attempted to implement the shuffling algorithm using a **while** loop, but the program contains several logical and runtime errors. Some important statements are also missing.

Problem Description

The program stores the following cards:

```
10 20 30 40 50 60 70 80
```

The shuffle should begin at the last index of the array. For each current index **i**, a random index **j** should be selected from **0** through **i**, including both endpoints.

If **i** and **j** are different, the values at those positions should be manually swapped using a temporary variable. A counter should keep track of the number of actual swaps performed.

After processing a position, the algorithm should move to the previous position and continue until the shuffle is complete.

The following program attempts to perform the shuffle:

```

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {

    const int SIZE = 8;
    int cards[SIZE] = {10, 20, 30, 40, 50, 60, 70, 80};

    srand(time(0));

    cout << "Original Cards:" << endl;

    for (int k = 0; k < SIZE; k++) {
        cout << cards[k] << " ";
    }

    cout << endl;

    int swaps = 0;
    int i = SIZE;

    while (i >= 0) {

        int j = rand() % i + 1;

        if (cards[i] = cards[j]) {

            int temp = cards[i];

            // Missing statement

            // Missing statement

```

```

        swaps += 1;
    }

    i++;
}

cout << "\nShuffled Cards:" << endl;

for (int k = 0; k < SIZE; k++) {
    cout << cards[k] << " ";
}

cout << "\nActual Swaps: " << swaps << endl;

return 0;
}

```

Find and fix all errors and complete the missing statements so that the program performs the required shuffle correctly.

Keep the overall structure of the program and use the existing `while` loop for the shuffling process. Perform all swaps manually using the existing temporary variable. Do not use `std::swap` or any built-in shuffle function.

Note: The shuffled order and number of actual swaps may differ between executions because random values are used.

Task 3: Security Camera Grid

A building is divided into a grid of rooms monitored by security cameras. Each room reports a security level from 0 to 9.

The security system can store information for at most **10 rows and 10 columns**.

Problem Description

Write a program that takes the number of rows and columns of the building and then inputs the security level of each room.

The program should only process a grid that can be stored by the security system. If the entered dimensions cannot be accommodated, display:

```
Invalid grid size.
```

and terminate the program.

For a valid grid, input and store the security level of every room. The program should then display the complete grid.

A room is considered **high-risk** if its security level is 7 or above. Report the total number of high-risk rooms.

The program must also determine whether **any row contains three or more high-risk rooms**. If such a row exists, print YES; otherwise, print NO.

Test Case 1

```

Input:
Rows = 4
Columns = 5

2 7 4 8 1
9 3 7 6 2
5 8 9 7 7
1 4 6 2 3

Output:
Security Grid:
2 7 4 8 1
9 3 7 6 2

```

```
5 8 9 7 7
1 4 6 2 3

High-Risk Rooms = 7
Three or More in a Row = YES
```

Test Case 2

```
Input:
Rows = 12
Columns = 5

Output:
Invalid grid size.
```

Task 4: The Number Table

A classroom activity generates a square number table. Each position in the table is based on the product of its row number and column number, but some products are replaced according to special rules.

Problem Description

Write a C++ program that takes an integer N from 3 to 9 and generates an N by N table.

For every position in the table, calculate:

$$value = row \times column$$

where both the row and column numbers begin from 1.

Display each position according to the following rules:

- If the value is divisible by both 3 and 5, display X.
- Otherwise, if the value is divisible by 3, display A.
- Otherwise, if the value is divisible by 5, display B.
- Otherwise, display the value itself.

While generating the table, also count how many positions were replaced by A, B, and X.

After the complete table has been displayed, print the three counts.

Test Case

```
Input:
N = 5

Output:
1 2 A 4 B
2 4 A 8 B
A A A A X
4 8 A 16 B
B B X B B

A Count = 6
B Count = 6
X Count = 2
```